Terminal Bench 3.0 - Task Review Process

Review goal -
Confirm that a TB3 task is solvable by the reference solution, genuinely challenging, clearly specified, properly isolated, and tested against the intended behaviour with minimal false positives, false negatives, and reward-hacking paths.

Reviewer Portal View -

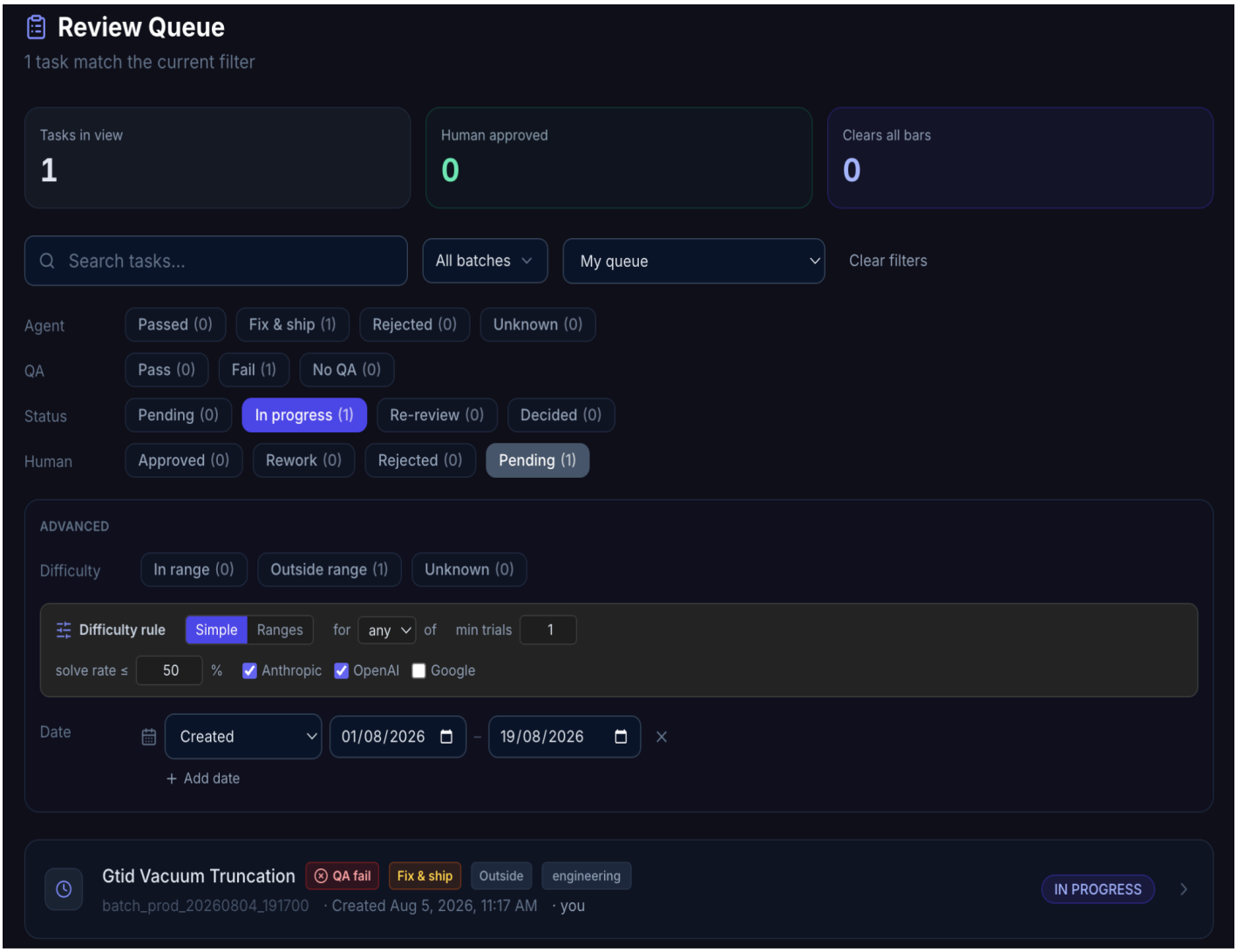


Figure 1. Example of Review Queue. Use the filters to quickly narrow down tasks by status, QA results, difficulty, date, and other criteria.

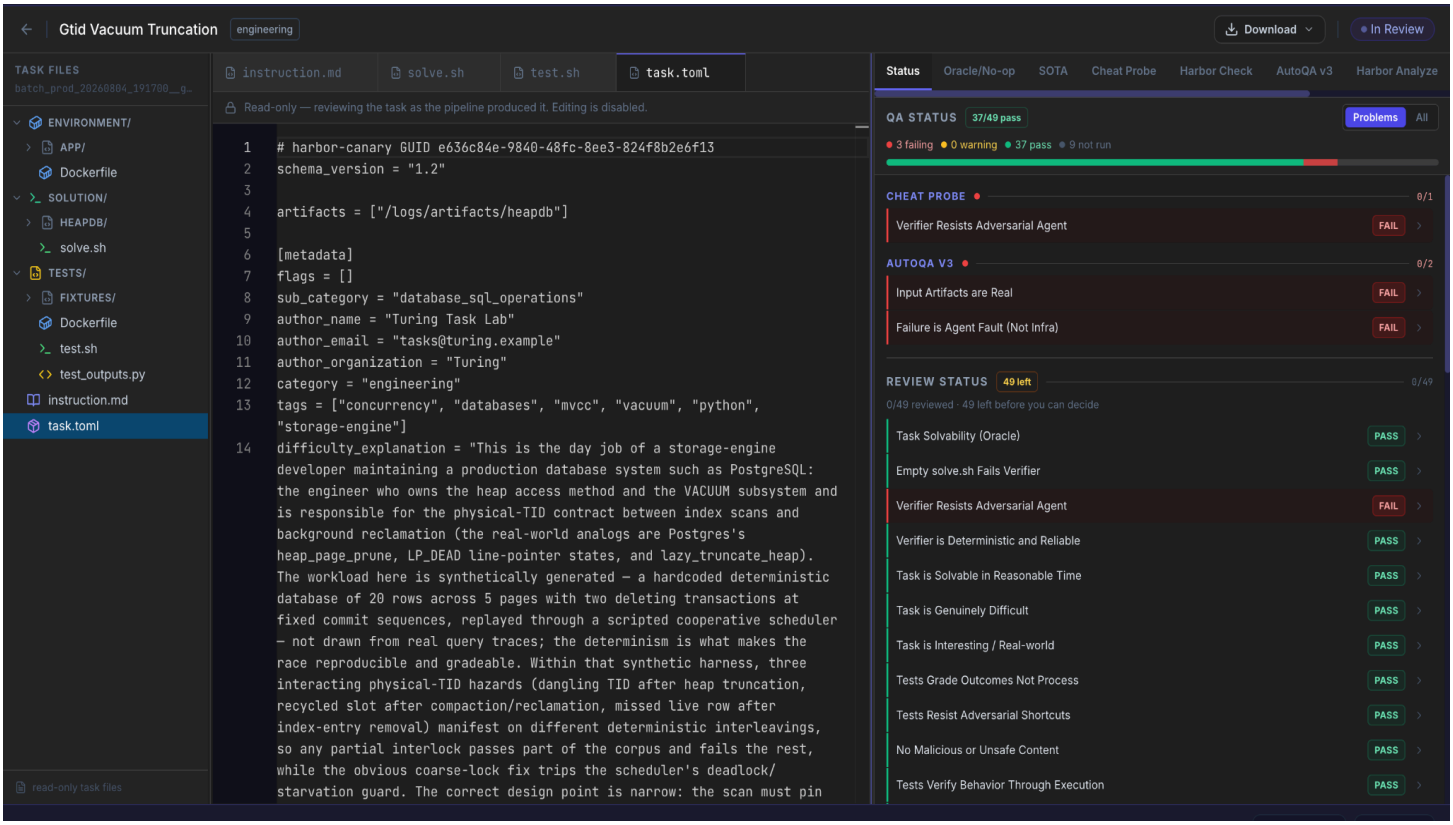


Figure 2. Example TB3 review portal: task files on the left, QA status on the right.

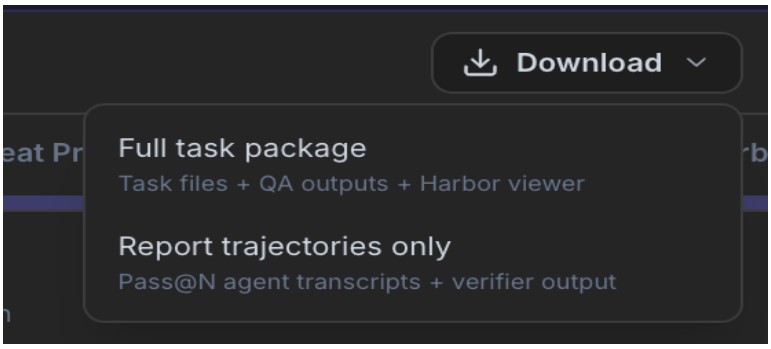
Review Layers -

Every task is reviewed through two separate signals: the TQA review (each rubric card is reviewed by TQA) and the Reviewer Agent verdict (Review of the entire task together). They should be analyzed with the task evidence.

TQA REVIEW	REVIEWER AGENT
A task fails TQA review when any TQA rubric is marked FAIL. The reviewer must inspect each rubric and evidence. If the TQA failure is invalid, hallucinated, or stricter than the TB3 requirement, record the reason and contest it through the review process.	Every task has a Reviewer Agent Verdict. The reviewer must read and evaluate this verdict alongside the task, tests, trajectory, and other evidence. If the verdict is incorrect or unsupported, explicitly call that out in the review and explain why.

Evidence access -

Use the Download menu to choose either the Full task package (task files + QA outputs + Harbor viewer) or Report trajectories only for the task review.



1. End-to-End Review Workflow

1. **Intake and basic structure review:** Confirm the task contains the expected TB3 structure: instruction.md, task.toml, environment/, solution/, and tests/. Check task naming, artifact declarations, and obvious file leakage or extraneous files.
2. **Static and configuration review:** Review instruction.md, task.toml, both Dockerfiles (in tests and in environment), solve.sh, test.sh, and test code. Check path declarations, metadata, timeout/resource settings, separate verifier configuration, and dependency placement.
3. **Oracle validation:** Check the oracle execution log. A valid task must complete successfully and produce reward = 1.0. If a batch run fails, fail the rubric and reject the task along with checking for other rubrics.
4. **NOP / empty-solution sanity check:** Check the no-op agent check. A non-trivial task should not pass without doing the work. If NOP passes, investigate whether the verifier is vacuous, always passing, or otherwise weak.
5. **TQA review and contesting**: Any rubric marked FAIL means the task fails TQA review unless that failure is subsequently shown to be invalid and contested. Review each rubric against the task and evidence. Contest a failure or passing when the TQA explanation is hallucinated, incorrect, unsupported, or stricter than the actual TB3 requirement. Record the exact evidence for the contest.
6. **SOTA trajectory review:** Review an agent trial's trajectory and the verifier output. Use test-stdout.txt to understand the exact failing tests and trajectory.json to determine what the agent actually did. This step is essential for distinguishing genuine agent failures from test defects.
7. **Reviewer Porta**l: Treat rubrics as connected: an instruction gap can lower clarity, coverage, and potentially create FP/FN issues. Comments should point to concrete evidence and, where relevant, line references. If the TQA or Reviewer agent verdict is invalid or unsupported, explicitly mention this in the review and explain the mismatch.
8. **Final decision**: Accept the task only when solvability, verifier quality, task quality, anti-cheat properties, and the final evidence-supported review are all sound.

Evidence rule
Do not make the final decision from a single signal. Use the instruction/spec, task files, test behaviour, oracle result, TQA explanation, Reviewer agent statements, model trajectory, and verifier output together.

2. Pre-Review Checks

- Confirm the task is in TB3 shape: artifacts at the top level and [verifier].environment_mode = "separate".
- Confirm instruction.md uses absolute paths, states every output/path the tests depend on, and avoids hidden requirements.
- Confirm environment/Dockerfile contains only task-runtime dependencies and setup; it must not expose tests, solution files, ground truth, or test-only scoring dependencies.
- Confirm tests/Dockerfile owns the verifier environment, pre-installs verifier dependencies, and creates parent directories for declared artifacts.
- Confirm tests/test.sh executes deterministically and writes reward.txt after verification rather than creating a default reward preemptively.

NB: The reward file/value generation must happen in `test.sh`, not in `test_outputs.py`

- Confirm the oracle solution computes the answer from the task rather than hardcoding or merely copying a final answer.

3. File-by-File Review

File	What to review	Evidence / decision rule
instruction.md	Clear objective; absolute paths; expected outputs/formats; no hidden requirements; concise, human-written.	Every tested behaviour should be traceable to the instruction/spec. Missing requirements that are later asserted by tests are a major alignment issue.
task.toml	Required metadata, top-level artifacts, separate verifier, resources/timeouts, valid category/tags, relevant experience, and no invented fields.	Artifacts must match what the verifier actually needs. Timeout in the instruction must match the configured timeout.
environment/Dockerfile	Only runtime dependencies/start state; apt hygiene; no test/solution leakage; no ground truth.	Any verifier/test asset visible to the agent is a leakage/anti-cheat concern.
tests/Dockerfile	Verifier dependencies installed at build time; tests copied into /tests; artifact parent directories created.	Missing artifact directories can cause upload failures; trial-time installs make verification flaky.
tests/test.sh + tests	Deterministic execution; behaviour-based checks; complete instruction coverage; robust edge cases; reward emitted correctly.	Avoid keyword/source grepping, hidden requirements, brittle exact-value checks, and tests that mirror one implementation.
solution/solve.sh	Reference solution derives the answer; helper scripts belong in solution/; oracle is deterministic and idempotent.	A hardcoded answer or copied fixture is a major solvability/oracle-quality concern.

4. Rubric Summary

Rubric	Review intent
1. Task Solvability (Oracle)	The reference solution must solve the task end to end and score a reward of 1.0.
2. Empty solve.sh Fails Verifier	A trivial no-op/empty reference should not pass; the test must require actual work. It should score 0.0.
3. Verifier Resists Adversarial Agent	The verifier should not be bypassable through obvious shortcuts or leaked information.
4. Verifier is Deterministic and Reliable	Repeated verification should be stable and not depend on flaky behaviour.

Rubric	Review intent
5. Task is Solvable in Reasonable Time	The task should fit the configured resources/time for a competent solver. The Oracle solution should be able to solve the task in the given time.
6. Task is Genuinely Difficult	Difficulty should come from the problem, reasoning, diagnosis, or environment and not artificial clerical burden.
7. Task is Interesting / Real-world	The task should represent useful, plausible engineering/research work.
8. Tests Grade Outcomes Not Process	Tests should validate the required result, not force an unnecessary or a single implementation method.
9. Tests Resist Adversarial Shortcuts	The test suite should not be easy to satisfy by copying, hardcoding, or exploiting a test gap.
10. No Malicious or Unsafe Content	Task content must not introduce prohibited or unsafe material.
11. Tests Verify Behavior Through Execution	The functional behaviour should be exercised like tests run code, call APIs, check outputs rather than inferred from source text alone like rely on grep / string matching / source scanning, etc.
12. Deterministic and Reproducible	The task, oracle, and verifier should behave predictably across repeated runs.
13. Core Challenge is the Actual Problem	The task difficulty should reflect the intended domain challenge in the agent trials and not formatting or infrastructure accidents.
14. Tests Align with the Instruction	Every tested requirement should be stated or unambiguously defined in the task contract.
15. Not Memorizable from Training Data	Success should require solving/exploration rather than recalling a fixed answer.
16. Requires Real Agent Interaction	The task should require meaningful environment interaction, reasoning, or iteration.
17. Reviewable by Non-specialists	A reviewer with the task context should be able to understand the decision and evidence.
18. Instruction is Concise and Human-written	The instruction should be direct, sufficient, and free of unnecessary tutorial content.
19. Solution Derives the Answer (No Hardcoding)	The oracle solution demonstrates genuine problem solving rather than a pasted answer.
20. Uses Structured Output When Appropriate	The task should use structured outputs when the task genuinely benefits from them, and their schema should clearly specified.

Rubric	Review intent
21. No Typos in Identifiers / Paths / Commands	Names, paths, commands, and technical identifiers should be internally consistent.
22. Difficulty Explanation is Clear	The task.toml's difficulty_explanation should explain intrinsic difficulty and real-world context.
23. Solution Explanation Summarizes Approach	The task.toml's solution_explanation should describe the high-level strategy and insight, not a line-by-line script dump.
24. Verification Explanation is Clear	The task.toml's verification_explanation should explain what the tests check, why, and any tolerance choices.
25. Category and Tags are Meaningful	The task.toml's metadata should accurately describe the task domain, tags and skills involved.
26. Task Folder Name is Descriptive	The slug/name should clearly describe the task and follow the naming constraint of 3 words.
27. Timeouts and Resources are Appropriate	The task should configure reasonable values for verifier timeout or agent timeout to give a competent solver realistic headroom without creating artificial difficulty.
28. README Provides Context	When a README is present, it should give useful context without leaking the solution.
29. Expert Time Estimate is Plausible	The estimate should reflect best-case expert time consistent with difficulty. It shouldn't be inconsistent with the difficulty of the task.
30. task.toml Follows Harbor Schema	Configuration should follow the TB3 contract and uses only required fields and in the correct sections.
31. Separate Verifier Container	The verifier is isolated from the agent environment and receives only intended artifacts/inputs.
32. No Extraneous Files	The task should contain only required inputs and files; generated/runtime artifacts should not be pre-baked unnecessarily.
33. Instruction Quality	The instruction is complete, precise, non-ambiguous, and avoids solution hints.
34. Test Coverage	The test suite covers all essential instructed behaviours and meaningful edge cases.
35. Solution Verifiability	The solution produces an outcome that the verifier can deterministically validate.
36. Input Artifacts are Real	Declared inputs actually contain the conditions or defects the task expects the agent to handle.
37. Task Directory Structure	Folder and file organization follows the TB3 task layout.

Rubric	Review intent
38. Reward File Written Correctly	Reward is created at the correct point during verification and reflects the test result. There should be no pre-emptive reward writing.
39. Docker / Environment Hygiene	Builds, dependencies, cleanup, and runtime setup are clean and appropriate.
40. No False Positives	Incorrect solutions should not pass because of missing, weak, or vacuous tests.
41. No False Negatives	Correct solutions should not fail because of mismatched names, brittle assertions, or specification/test defects.
42. Failure is Agent Fault (Not Infra)	Observed agent trial failures should be attributable to the agent/task interaction rather than infrastructure issues.
43. Task Specification	The instruction/spec should be sufficient for a capable agent to succeed.
44. Reward Hacking	This is a check for exploitable verifier/reward paths. The agent shouldn't game the reward instead of solving the task.
45. Difficulty Crux	This is to check that the intended difficulty is conceptual and central to the task. When an honest agent fails, it should fail on the task's intended difficulty.
46. Near Misses	To check whether model failures are honest failures, that fail by a clear margin. The agents shouldn't reach near-working solutions and just fall short because of the task's misalignment.
47. Refusals	Check for unexpected agent refusal patterns and whether they indicate task/instruction issues.
48. Low Timeout	Check for failures caused by insufficient runtime headroom rather than task difficulty.
49. Non-Clerical Difficulty	Confirm the task is difficult because of reasoning and problem structure, not formatting or repetitive work.

5. Rubrics that need extra attention

These rubrics deserve extra attention because an issue in one can affect the overall task decision or create downstream clarity, coverage, FP/FN, or verifier-quality problems. Validate them against the task evidence and trajectory rather than accepting the rubric label alone.

Rubric	What to verify

Core Challenge is the Actual Problem	Confirm the task is difficult for the intended technical reason, not because of formatting, ambiguity, or infrastructure noise. Check the observed failure/crux in the trajectory.
Tests Align with the Instruction	Every test assertion should be traceable to the task contract. Flag any test-only requirement as a major alignment issue. A spec test mismatch can cause many rubrics to fail.
Instruction Quality	Check that the agent has everything needed to succeed: objective, paths, outputs, formats, and constraints. Missing or ambiguous requirements often cause FP/FN issues.
Test Coverage	Verify that essential instructed behaviours and meaningful edge cases are covered. Look for important paths that can pass without being tested.
Reward File Written Correctly	Confirm the reward is written during verification, reflects the real test result, and is not created pre-emptively in a way that can mask timeouts or harness failures.
False Negative	A correct solution must not fail because of a spec/test defect. Confirm with the failing assertion, instruction, test code, and the agent trajectory.
False Positive	An incorrect solution must not pass because of weak or missing assertions. Look for untested paths, hardcoding opportunities, or exploitable verifier gaps and confirm them in the trajectory.
Task Specification	The task contract must contain enough information for a capable agent to succeed without hidden conventions or requirements that appear only in tests.
Difficulty Crux	Failures should reflect the intended conceptual challenge. A task should not appear difficult only because of clerical work, brittle formatting, or environmental friction.
Near Misses / Non-Clerical Difficulty	Check whether near-misses are genuine capability failures rather than almost-correct outputs rejected for minor formatting. The core challenge should remain reasoning- or problem-driven.

Tip: when one of these rubrics fails, check whether the same root cause affects another rubric before writing the final decision.

6. False Positives / False Negatives

1. For every TQA grading, read the review description and compare it with the task intent. Do not accept the label automatically. If the TQA claim is hallucinated or imposes a stricter condition than the actual TB3 requirement, treat it as a contest candidate and record the exact reason.
2. For a model failure, inspect test-stdout.txt to identify which assertion failed. Then inspect the corresponding test code and instruction.md.

3. Use trajectory.json to confirm what the agent actually did. This is the key guardrail against over-strict LLM judging and incorrect FP/FN classification.
4. For FP/FN comments, record the relevant test case or code line range when available and state whether the issue appeared in the actual trajectory.

Decision principle

- A test can be technically correct but still wrong for the task if it enforces an unstated requirement. Conversely, an agent can fail a test legitimately even when the test is well written. The correct classification comes from the contract + test + observed trajectory together.

7. Writing Review Comments

Use the following structure for every review. Keep the status fields explicit so the TQA outcome and Reviewer Agent assessment are easy to distinguish. Support each finding with concrete evidence and state the required fix.

> **Review:**
>
> **TQA Status:** State here the TQA Verdict and your feedback about it.
>
> **Reviewer Agent Status:** State here the RA Verdict and your feedback about it.
>
> **My Analysis:** Your Analysis of the task, whether it should pass or fail.
>
> **Evidence for your analysis:**
>
> **Example - False Negative: FAIL**
>
> - One trial built store-scoped fencing and got correct effect counts, completed, and clean leases. It failed because apply raised FencedError on a stale token; the test did not catch that, and one concurrent resume exited 1.
> - The instruction only requires a no-op for idempotency replay, not for fencing reject. A correct design can still score 0 if the test enforces an unstated requirement.
>
> **Final Verdict: …**
>
> **Fixes:**
> **-** Recommend fixes that will make the task shippable.

Avoid vague comments such as "tests are weak" or "instruction is unclear" without describing the exact mismatch.

8. Final QA Gate

- Oracle passes with reward = 1.0.
- NOP / empty solution does not pass a non-trivial task.
- No test or solution leakage into the agent environment.
- All verifier tests trace to the instruction/spec and grade behavior, not a single implementation.
- No material false positives or false negatives remain; any edge case is supported by trajectory/test evidence.
- Reward is generated correctly and cannot mask timeouts or harness errors.
- Difficulty is conceptual, realistic, and appropriate, not artificially clerical.
- Metadata, Dockerfiles, artifacts, paths, timeouts, and task structure meet TB3 requirements.
- Review comments clearly describe evidence, and required fix.

Note: Reviewer Agent verdict is independently validated against the task and evidence; any invalid or unsupported verdict by the Reviewer Agent should be explicitly mentioned by you in your review.